

Google Cloud Pub/Sub

Index

1. Introduction
 - 1.1 What is Pub/Sub?
2. Why Do We Use Pub/Sub?
3. When Should We Use Pub/Sub?
4. Pub/Sub Architecture
5. **Core Pub/Sub Components**
 - Publisher
 - Topic
 - Subscription
 - Message
 - Message Attributes
 - Subscriber
 - How Message Flow Works
6. **Message Delivery and Reliability**
 - ACK – Acknowledgement**
 - ACK Deadline
 - What Happens When ACK Is Not Received?
7. **Message Retry and Duplicate Messages**
 - Retry Policy
 - Exactly-Once Delivery
8. **Pull and Push Subscriptions**
9. **Dead-Letter Topics**
10. **Message Management**
 - Message Retention**
 - Topic Retention

-Subscription Retention

11. Message Ordering

-Ordering Keys

12. Subscription Filters

13. Pub/Sub Schemas

-What is a Schema?

-Avro

-Protocol Buffers (Protobuf)

-Why schemas are used

-Schema validation

14. Pub/Sub Security and IAM

15. Pub/Sub Pricing – Basic Understanding

16. Pub/Sub Integration with GCP Services

17. Monitoring and Troubleshooting

18. Common Problems

19. Pub/Sub vs Kafka vs Event arc vs Cloud Tasks

20.Explanation about the UI-Interface in PUB/SUB

21.Explanation about the UI-Interface in PUB/SUB Topics

22.Step-by-Step UI Guide to Create a Pub/Sub Topic

23.Explanation about the UI-Interface in PUB/SUB Subscription

24.Step-by-Step UI Guide to Create a Pub/Sub Subscriptions

25. Conclusion

1. Introduction

What is Google Cloud Pub/Sub?

Google Cloud Pub/Sub is a fully managed messaging service provided by Google Cloud.

It allows applications and services to **send and receive messages asynchronously** without communicating directly with each other.

Simple example:

Publisher



Pub/Sub



Subscriber

The publisher sends a message to Pub/Sub, and the subscriber receives and processes that message.

2. Why Do We Use Pub/Sub?

Pub/Sub is mainly used for:

- Asynchronous communication
- Decoupling applications
- Real-time event processing
- Streaming data pipelines
- Connecting multiple services
- Handling high volumes of messages
- Building event-driven applications
- **Decoupling (Loose Coupling):** Producers and consumers don't need to know each other's location, network protocols, or availability.
- **Auto-Scaling with Zero Ops:** Scales from a few messages a day to millions per second with zero server provisioning or cluster management.
- **Global Availability:** Operates as a single global system. You publish to a single endpoint anywhere in the world, and subscribers receive messages globally with low latency.

- Resilience: Pub/Sub is designed as a highly reliable and scalable messaging service with replicated infrastructure and mechanisms that support message delivery and recovery.

Example

Suppose an e-commerce application creates an order.

Instead of directly calling multiple services:

Order Application:

```
+----> Payment
+----> Inventory
+----> Email
```

We can use Pub/Sub:

Order Application

```
|
Pub/Sub Topic
|
+----> Payment
|
+----> Inventory
|
+----> Email
```

This makes the system more flexible and loosely coupled.

3. When Should We Use Pub/Sub?

Use Pub/Sub when:

- Applications need to communicate asynchronously.
- Large amounts of events need to be processed.
- Multiple applications need the same event.
- You are building real-time data pipelines.
- You need to separate producers and consumers.
- You need an event-driven architecture.

- Ingesting high-throughput data (IoT telematics, clickstream analytics, user logs).
- Fan-out architectures where one event triggers multiple independent downstream microservices.
- Buffering workloads to protect slower backend databases from sudden traffic spikes.
- Event-driven architectures (triggering ETL jobs, notifications, or serverless functions).

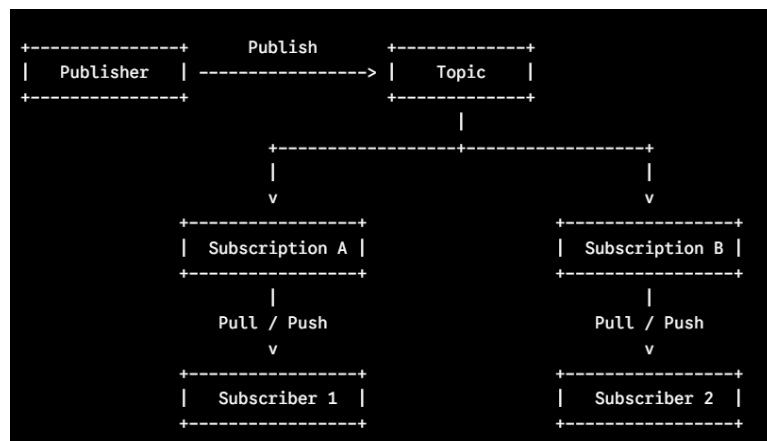
Common Examples

- E-commerce order processing
- IoT data ingestion
- Application logs
- Real-time analytics
- Notifications
- ETL/ELT pipelines
- Microservices communication

Real-World Use Cases

- **E-Commerce Order Pipeline:** When an order is placed, a single event is published. Three independent subscriptions process it concurrently: Payment Service, Inventory Reservation, and Email Notification.
- **Real-time Fraud Detection:** Ingests banking transaction streams into Cloud Dataflow for sub-second anomaly scoring before persisting to BigQuery.
- **Smart Grid / IoT Ingestion:** Hundreds of thousands of smart meters stream telemetry every 5 seconds into Pub/Sub topics for aggregation.

4. Pub/Sub Architecture: The basic Pub/Sub architecture is:



5. Important Pub/Subcomponents

Component	Description
Message	The payload (bytes or JSON) along with optional key-value metadata attributes.
Topic	A named resource representing a feed/channel to which messages are sent.
Publisher	The application or service that creates and sends messages to a topic.
Subscription	A named resource representing a stream of messages from a specific topic to be delivered to an application.
Subscriber	The application or service that receives and processes messages from a subscription.

Easy Way to Remember

Publisher → Topic → Subscription → Subscriber

Publisher

A **publisher** is an application or service that sends messages to a Pub/Sub topic.

Example:

```
{  
  "order_id": "ORD101",  
  "amount": 500  
}
```

The publisher sends this message to the topic.

Topic

A **topic** is a named resource where publishers send messages.

Example:

Topic:

ecommerce-orders

Publisher:

Order Application

|

v

ecommerce-orders

A topic can have multiple subscriptions.

Subscription

A **subscription** is used by a subscriber to receive messages from a topic.

Example:

Topic

|

+---- Subscription A → Application A

|

+---- Subscription B → Application B

Multiple subscriptions allow different applications to independently process the same published event.

Important

A subscriber consumes messages from a **subscription**, not directly from the topic.

Message

A message is the actual data sent through Pub/Sub.

Example:

{

"customer_id": 101,

"event": "order_created",

"amount": 500

$$\}$$

A message can contain:

- Data
- Attributes
- Message metadata

Subscriber

A **subscriber** is an application or service that receives and processes messages from a subscription.

Examples:

- Python application
- Cloud Run
- Cloud Run function
- Dataflow
- Other supported services

How Message Flow Works

The Message Flow

1. **Publish:** The publisher pushes a message to projects/{project-id}/topics/{topic-name}.
2. **Replication & Routing:** Pub/Sub writes the message to storage across multiple zones and routes a copy to every attached subscription.
3. **Delivery:** The subscription delivers the message to an available subscriber instance.
4. **Acknowledgement (ACK):** The subscriber processes the message and sends an ACK response back to Pub/Sub.
5. **Deletion:** Pub/Sub removes the acknowledged message from that specific subscription's queue.

[illegible]

Step-by-Step

1. Publisher creates a message.
2. Publisher sends the message to a topic.
3. Pub/Sub makes the message available through the subscription.
4. Subscriber receives the message.
5. Subscriber processes the message.
6. Subscriber sends an ACK after successful processing.

6.Message Delivery and Reliability

ACK – Acknowledgement

ACK means acknowledgement.

After successfully processing a message, the subscriber sends an ACK to Pub/Sub.

Pub/Sub

|

| Message

v

Subscriber

|

| Processing successful

v

ACK

If the message is not successfully acknowledged, Pub/Sub can make the message available for redelivery.

ACK, NACK, and Ack Deadline

- **Ack Deadline:** The window of time (default: 10s, max: 600s) Pub/Sub waits for an acknowledgment before assuming processing failed.
- **ACK:** Confirms success. The message will not be delivered again on this subscription.
- **NACK:** Explicitly tells Pub/Sub processing failed, triggering immediate redelivery.

- **Ack Deadline Extension:** The Google Cloud client libraries automatically extend the deadline while your code is actively processing the message

7. Message Retry and Duplicate Messages

Pub/Sub commonly uses **at-least-once delivery**.

This means a message can sometimes be delivered more than once.

Example:

Message

↓

Subscriber

↓

Processing

↓

ACK fails/delayed

↓

Message may be delivered again

Therefore, applications should be designed to handle duplicate messages safely.

8. Pull and Push Subscriptions

Pub/Sub supports different ways of delivering messages.

Pull Subscription

The subscriber requests messages from Pub/Sub.

Subscriber

|

| Pull

v

Pub/Sub

|

| Message

v

Subscriber

Commonly used for worker applications and streaming consumers.

Push Subscription

Pub/Sub sends the message to an HTTP endpoint.

Pub/Sub

|

| HTTP Request

v

Cloud Run / HTTP Service

Useful for HTTP-based applications and serverless services.

Comparison

Feature	Pull	Push
Message retrieval	Subscriber requests	Pub/Sub sends
HTTP endpoint	Not required	Required
Consumer control	Higher	Lower
Common use	Workers/Dataflow	Cloud Run/HTTP services

9. Dead-Letter Topics

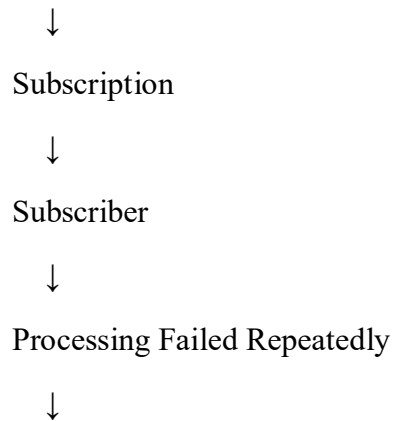
A **Dead-Letter Topic** is used to handle messages that repeatedly fail processing.

If a malformed payload crashes your consumer, it can loop indefinitely (poison pill).

- A **Dead-Letter Topic** isolates these messages after a specified number of failed delivery attempts (between 5 and 100).
- Unlocks easier debugging without holding up valid message traffic.

Example:

Main Topic



Dead-Letter Topic

Dead-letter topics help isolate problematic messages instead of continuously retrying them.

Common reasons for failure:

- Invalid data
- Missing fields
- Application errors
- Unexpected message format

10. Message Management

Message retention determines how long Pub/Sub can retain messages according to the configured retention settings.

Message Retention: Unacknowledged messages can be retained for **10 minutes up to 31 days** (default: 7 days). Topics can also retain messages independently.

Retention can be useful when:

- A subscriber is temporarily unavailable.
- Messages need to be replayed.
- An application needs to recover from a failure.

Retention settings can be configured at the appropriate Pub/Sub resource level.

11. Message Ordering

By default, applications should not assume that messages will always be processed in a particular order.

Message Ordering: By default, Pub/Sub processes messages out of order for maximum parallelism. To preserve order:

- Enable **Message Ordering** on the subscription.
- Supply an `ordering_key` during message publication (e.g., `customer_id_1029`). All messages with the same key are processed sequentially.

If ordering is required, Pub/Sub supports **ordering keys**.

Example:

Ordering Key: customer-101

Message 1 → Order Created

Message 2 → Payment Completed

Message 3 → Order Shipped

Ordering should only be used when the application actually requires ordered processing because it can affect parallelism.

12.Subscription Filters

What are Subscription Filters?

A **subscription filter** is a rule that tells Google Cloud Pub/Sub which messages a subscription should receive.

It checks the **message attributes** and delivers only the messages that match the filter.

Example:

A topic contains order messages from different regions. A subscription can be configured to receive only messages where:

`region = "India"`

Why Use Subscription Filters?

- **Save Compute and Network:**
Only matching messages are delivered to the subscriber. Unwanted messages do not need to be processed by the application.
- **Easy Message Routing:**
One topic can contain different types of messages, and different subscriptions can receive only the messages they need.

- **No Custom Code:**

You do not need to write filtering logic inside your application, Cloud Run service, or function.

How Subscription Filters Work

1. A publisher sends messages to a topic.
2. Each message can contain **attributes** such as region, priority, or type.
3. Pub/Sub checks the subscription filter.
4. **If the message matches:** Pub/Sub delivers it to the subscriber.
5. **If the message does not match:** Pub/Sub does not deliver it to the subscriber.

13. Pub/Sub Schemas

- What is a Schema?
- Avro
- Protocol Buffers (Protobuf)
- Why schemas are used
- Schema validation

Schema in Google Cloud Pub/Sub

What is a Schema?

A **schema** defines the structure and format of a message in Pub/Sub.

It tells us:

- What fields the message should have.
- What data type each field should have.
- What format the message should follow.

When a schema is attached to a topic, Pub/Sub checks messages before accepting them.

Example:

If amount should be a number:

```
{  
  "order_id": "ORD101",
```

```
"amount": 500.50
}
```

This is valid.

If the publisher sends:

```
{
  "order_id": "ORD101",
  "amount": "five hundred"
}
```

The message does not follow the expected data type.

Schema Formats

Pub/Sub supports two main schema formats:

Feature	Apache Avro	Protocol Buffers (Protobuf)
Definition Language	Defined using JSON format (.avsc).	Defined using Google's Interface Definition Language (.proto).
Payload Encodings	Supports binary encoding and JSON encoding.	Supports binary wire format and JSON encoding.
Primary Ecosystem	Widely used across Apache Spark, Hadoop, Kafka, and big data pipelines.	Standard for Google Cloud services, gRPC, and microservice APIs.
Schema Evolution	Supports default values, aliases, and type unions for forward/backward compatibility.	Evolves by adding tagged field numbers without breaking older consumers.

Apache Avro (order.avsc):

```
{
  "type": "record",
```

```
"name": "OrderEvent",
"fields": [
  {"name": "order_id", "type": "string"},
  {"name": "customer_id", "type": "string"},
  {"name": "amount", "type": "double"},
  {"name": "is_completed", "type": "boolean", "default": false}
]
```

Protocol Buffers (order.proto):

```
syntax = "proto3";
message OrderEvent {
  string order_id = 1;
  string customer_id = 2;
  double amount = 3;
  bool is_completed = 4;
}
```

Why Are Schemas Used?

- **Prevent Invalid Messages:**
Schemas help stop incorrect or incomplete messages from entering Pub/Sub.
- **Common Data Format:**
A schema provides a common format between the **publisher and subscriber** so both sides know what data to expect.
- **Improve Data Quality:**
It ensures that data sent to systems like **BigQuery** follows the expected structure and data types.
- **Reduce Message Size:**
Avro and Protobuf can use binary formats, which can make messages smaller than normal JSON.

How Schema Validation Works

- 1. Create a Schema:**
Define the fields and data types for the message.
- 2. Attach Schema to Topic:**
Attach the schema to the Pub/Sub topic.
- 3. Publisher Sends Message:**
The publisher sends a message to the topic.
- 4. Pub/Sub Validates the Message:**
Pub/Sub checks whether the message follows the schema.
- 5. Valid Message:**
If the message follows the schema, Pub/Sub accepts it.
- 6. Invalid Message:**
If the message does not follow the schema, Pub/Sub rejects it with an `INVALID_ARGUMENT` error.

14. Pub/Sub Security and IAM

Google Cloud uses **IAM (Identity and Access Management)** to control access to Pub/Sub resources.

Core IAM Roles

Role Name	Role ID	Permissions
Pub/Sub Admin	roles/pubsub.admin	Full control over topics, subscriptions, and schemas.
Pub/Sub Publisher	roles/pubsub.publisher	Publish messages to specific topics (pubsub.topics.publish).
Pub/Sub Subscriber	roles/pubsub.subscriber	Pull/consume messages, acknowledge, and modify ack deadlines.
Pub/Sub Viewer	roles/pubsub.viewer	Read metadata and configuration without payload access.

15. Pub/Sub Pricing – Basic Understanding

Data Volume Ingestion/Delivery: Billed per TiB of message volume (first 10 GiB/month is free).

Cross-Region / Internet Egress: Standard network egress rates apply if consuming messages from outside the topic's region or over the public internet.

Snapshots & Extended Retention: Retaining unacknowledged or acknowledged messages beyond baseline incurs standard regional storage fees.

Pub/Sub pricing can depend on factors such as:

- Amount of data processed
- Message volume
- Message retention
- Storage
- Network usage

16. Pub/Sub Integration with GCP Services

Pub/Sub + Cloud Run Functions

Used for event-driven serverless processing.

Pub/Sub



Cloud Run Function



Processing

Pub/Sub + Cloud Run

Used when a containerized application needs to process Pub/Sub events.

Pub/Sub



Cloud Run



Application

Pub/Sub + Dataflow

Used for complex streaming data processing.

Pub/Sub



Dataflow



Transform



BigQuery

Dataflow can perform:

- Filtering
- Transformation
- Aggregation
- Windowing
- Streaming processing

Pub/Sub + BigQuery

Pub/Sub can be used to ingest streaming events into BigQuery.

A BigQuery subscription can write suitable Pub/Sub messages directly into BigQuery.

Pub/Sub



BigQuery Subscription



BigQuery

For more complex transformations:

Pub/Sub



Dataflow



BigQuery

17. Monitoring and Troubleshooting

Use **Cloud Monitoring** and **Cloud Logging** to monitor Pub/Sub applications.

Important things to check:

- Message backlog
- Unacknowledged messages
- Subscriber errors
- Function errors
- Permission errors
- Message delivery
- Retry behavior

18. Common Problems

Problem	Possible Cause
Message not received	Subscription or consumer problem
Message repeatedly delivered	ACK/processing failure
Permission denied	Missing IAM permission
Function not triggered	Trigger configuration problem
BigQuery insert failed	Schema/IAM/data problem
Message backlog increasing	Consumer is slower than publisher

19. Pub/Sub vs Kafka vs Eventarc vs Cloud Tasks

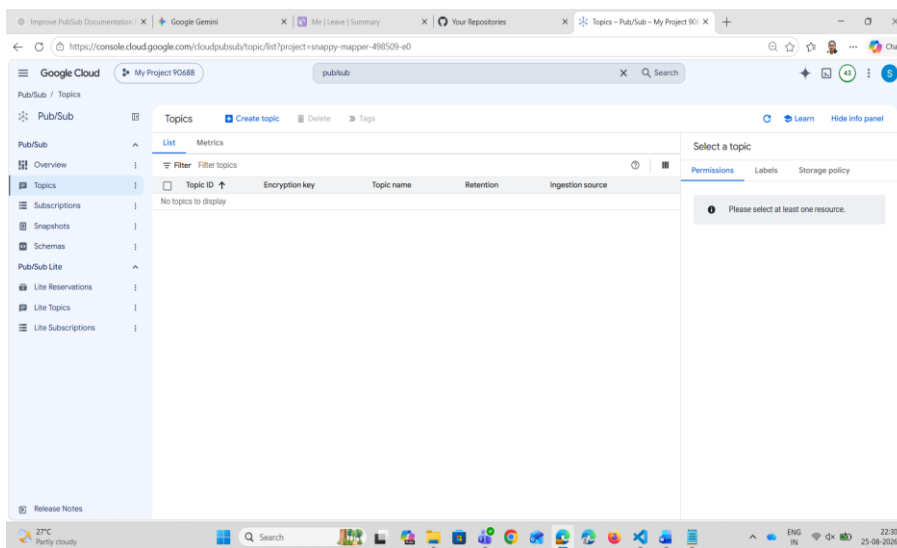
Feature	Pub/Sub	Kafka	Eventarc	Cloud Tasks
Main purpose	Messaging/event ingestion	Event streaming	Event routing	Task execution
Management	Fully managed	Managed/Self-managed	Fully managed	Fully managed
Common use	Streaming/event-driven systems	Large event-streaming platforms	GCP event routing	Background tasks

Feature	Pub/Sub	Kafka	Eventarc	Cloud Tasks
Ordering	Ordering keys	Partitions	Not the primary purpose	Not the primary purpose
Scheduling	Limited compared with task queues	Not its main purpose	Event routing	Strong scheduling support
Best for	GCP messaging pipelines	Kafka ecosystem and event streams	Connecting events to services	Reliable task execution

Troubleshooting Checklist

- **Unacked Message Backlog Growing:** Increase consumer instances, increase worker concurrency, or extend the Ack Deadline.
- **Duplicate Message Flood:** Verify your subscribers acknowledge messages before the deadline expires.
- **Permission Denied (403):** Ensure the publisher has roles/pubsub.publisher on the topic and the subscriber has roles/pubsub.subscriber on the subscription.

20.Explanation about the UI-Interface in PUB/SUB:



Top Navigation & Action Buttons(from above image):

- **+ Create topic:** Opens the configuration form to create a new message feed.

- **Delete:** Removes selected topics. *When to use:* Cleaning up unused resources. Note that deleting a topic will orphan any attached subscriptions.
- **Tags:** Key-value metadata attached to GCP resources for centralized organization and resource access management.
- **Metrics tab:** Shows real-time Cloud Monitoring graphs (published volume, message counts, byte rates) instead of the resource list.
- **Hide info panel / Info Panel (Right Side):** Displays access controls and details for the selected topic:
 - **Permissions:** Assigns IAM roles (e.g., Pub/Sub Publisher, Pub/Sub Admin) to users or service accounts.
 - **Labels:** Simple key-value pairs used for internal tracking and billing breakdowns.
 - **Storage policy:** Restricts the geographic Google Cloud regions where messages are allowed to be physically stored.

Table Columns

- **Topic ID:** The short unique identifier for the topic within the project (e.g., `iot-telemetry`).
- **Encryption key:** Shows whether the topic uses a Google-managed key or a customer-managed KMS key (CMEK).
- **Topic name:** The fully qualified GCP path format: `projects/{project-id}/topics/{topic-id}`.
- **Retention:** Indicates if topic-level message retention is enabled (keeps messages even if no subscription exists yet).
- **Ingestion source:** Shows whether the topic automatically pulls data from external sources (such as AWS Kinesis, Cloud Storage, or Azure Event Hubs).

This is the left-hand navigation sidebar for Google Cloud Pub/Sub, divided into standard **Pub/Sub** and **Pub/Sub Lite**.

1. Overview

- **What:** The central dashboard for your Pub/Sub service.
- **Why used:** To quickly monitor the overall health and traffic of your messaging pipelines.
- **How:** Aggregates real-time charts (total messages published, unacknowledged backlogs, throughput, and error rates).

- **When:** When you log in to check system performance or spot sudden traffic spikes.

2. Topics

- **What:** Named channels (feeds) where senders (publishers) push messages.
- **Why used:** To collect incoming data and broadcast it to multiple listening services.
- **How:** Applications send data payloads (JSON, text, or bytes) to a specific topic endpoint.
- **When:** Whenever you start a new messaging pipeline (e.g., creating an orders or sensor-data topic).

3. Subscriptions

- **What:** Named queues attached to a topic that receive and hold messages for readers (subscribers).
- **Why used:** To consume data. Every service that needs to read from a topic must have its own subscription.
- **How:** Delivers messages to your app either by your app asking for them (**Pull**) or Pub/Sub sending an HTTPS request (**Push**).
- **When:** Whenever you build a downstream worker, Cloud Function, or pipeline that processes topic data.

4. Snapshots

- **What:** A point-in-time capture of the exact state of a subscription (which messages are acknowledged vs. unacknowledged).
- **Why used:** To replay historical messages or skip a massive backlog of unneeded data.
- **How:** Pub/Sub saves the acknowledgement state at a specific timestamp. You can "seek" a subscription back to this snapshot anytime.
- **When:** Useful during application deployments. If a bug crashes your consumer, you can fix the bug and seek back to the snapshot to reprocess lost messages.

5. Schemas

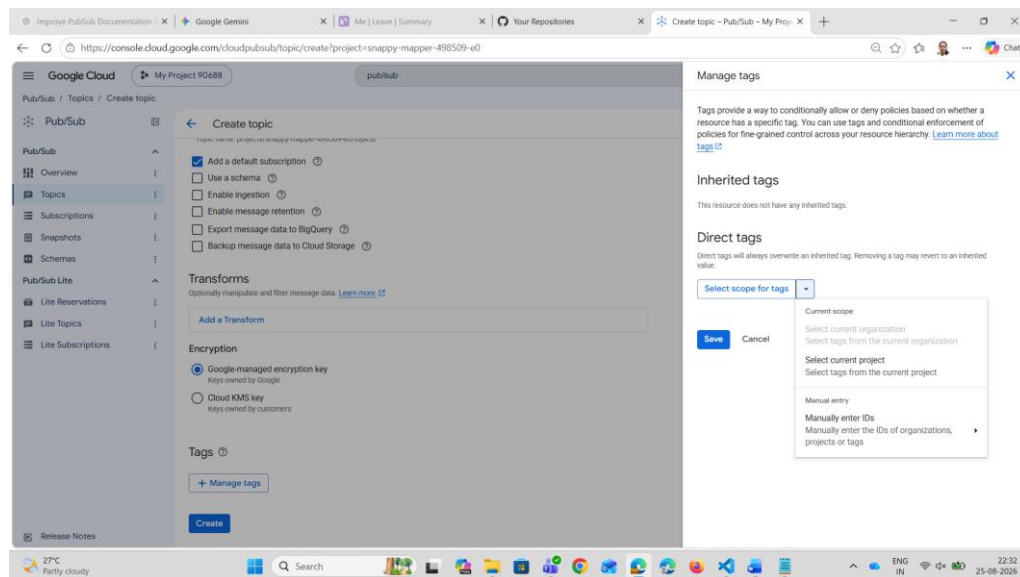
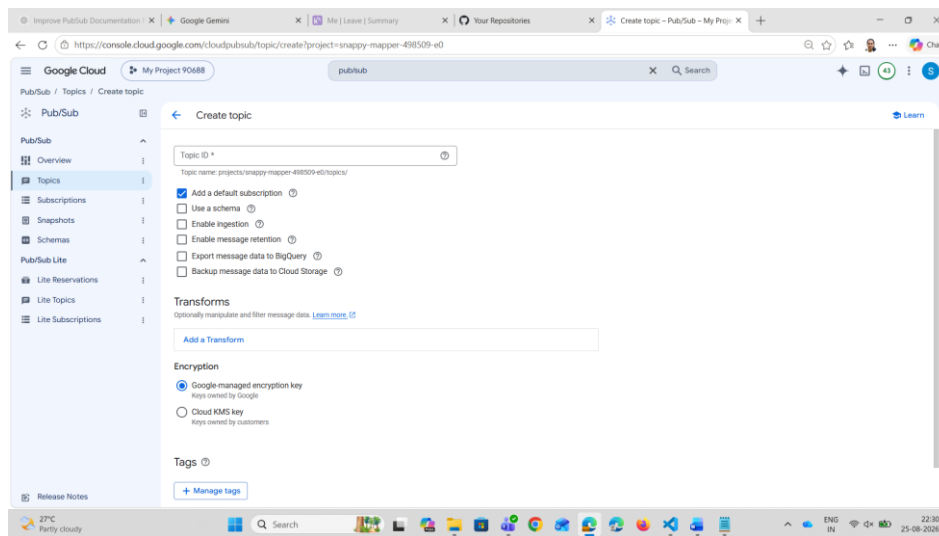
- **What:** Strict data format rules (contracts) written in **Avro** or **Protocol Buffers (Protobuf)**.

- **Why used:** To guarantee data quality and prevent bad data from crashing downstream apps.
- **How:** You attach a schema to a topic; Pub/Sub validates incoming messages and rejects any payload that doesn't match the required fields and types.
- **When:** When multiple teams produce data and you need strict structure enforcement (e.g., banking transactions, API payloads).

6. Pub/Sub Lite (Reservations, Lite Topics, Lite Subscriptions)

- **What it is:** A lower-cost, zonal version of Pub/Sub (similar to Apache Kafka) where you manually provision and manage storage and throughput capacity instead of using a fully serverless model.
- **Lite Reservations:**
 - **What/Why:** A pool of pre-purchased throughput capacity (MiB/s publish/subscribe) shared across multiple Lite topics within a region.
- **Lite Topics:**
 - **What/Why:** Partitioned message channels within a single zone where you explicitly configure throughput and retention storage.
- **Lite Subscriptions:**
 - **What/Why:** Partitioned reader queues linked directly to a Lite topic.
- **How:** Messages are divided across fixed partitions that you manage, trading serverless auto-scaling for significantly lower costs.
- **When:** When you have high-volume, predictable streaming workloads where you want to minimize costs and can manage capacity yourself.

21.Explanation about the UI-Interface in PUB/SUB Topics:



Core Configuration Fields & Options(from above 2 images)

- **Topic ID:** The name of the Pub/Sub topic. This is required.
- **Add a default subscription:** Automatically creates a subscription for the topic. This is useful for quick testing.
- **Use a schema:** Ensures that messages follow a defined format, such as Avro or Protocol Buffers. It helps prevent invalid data.

- **Enable ingestion:** Allows Pub/Sub to receive data directly from external services such as AWS Kinesis and Azure Event Hubs.
- **Enable message retention:** Keeps messages on the topic for a configured period. This allows messages to be replayed later.
- **Export message data to BigQuery:** Sends Pub/Sub messages directly to BigQuery for data analysis.
- **Backup message data to Cloud Storage:** Saves Pub/Sub messages directly to a Cloud Storage bucket.

Transforms (JavaScript UDF)

- **What it is:** A JavaScript function used to modify messages before they reach subscribers.
- **How it works:** It can filter or change message data during processing.
- **Example:** Remove unnecessary fields or filter test messages.
- **Disable transform:** Stops the transformation without deleting the code.
- **Validate / Test transforms:** Allows you to test the transformation with sample messages before using it.

Encryption

- **Google-managed encryption key:** The default option. Google manages the encryption keys and protects the data.
- **Cloud KMS key:** Allows you to use your own encryption key through Google Cloud KMS. This is useful when you need more control over encryption.

Manage Tags

What are Tags?

Tags are key-value pairs used to organize and manage Google Cloud resources.

Example:

- environment: production
- cost-center: analytics

Tags can also be used with **IAM and organization policies** to control access and apply security rules.

Why Are Tags Used?

- **Security:** Control who can access a resource.

- **Resource Management:** Organize resources based on environment or purpose.
- **Governance:** Apply common rules across projects and resources.
- **Compliance:** Help enforce company policies.

Types of Tags

Inherited Tags

- Tags that come from a parent resource such as a Project, Folder, or Organization.
- Example: If a project has environment: dev, resources inside the project can inherit this tag.

Direct Tags

- Tags added directly to the Pub/Sub topic.
- They apply specifically to that topic.

Select Scope for Tags

This option allows you to choose where the tags are managed:

- **Current Organization:** Use tags defined at the organization level.
- **Current Project:** Use tags defined for the project.
- **Manual Entry:** Enter the tag information manually.

Save / Cancel

- **Save:** Applies the selected tags to the topic.
- **Cancel / X:** Closes the panel without adding tags.

When to Use Tags

- Use tags mainly in **enterprise and production environments** where security and organization policies are important.

22.Step-by-Step UI Guide to Create a Pub/Sub Topic

Step 1: Open the Pub/Sub Topics Page

1. In the top search bar, type pub/sub and press **Enter**, or click the **Navigation menu** () in the top-left corner.
2. Under the **Pub/Sub** section on the left sidebar, click **Topics**.

Step 2: Click "Create topic"

1. At the top of the **Topics** list view, click the + Create topic button.
2. This opens the **Create topic** configuration page.

Step 3: Enter the Topic ID

1. In the **Topic ID** input box, enter a name for your topic (e.g., my-first-topic or ecommerce-orders).
2. Make sure the name uses lowercase letters, numbers, hyphens (-), or underscores (_).

Step 4: Configure Quick Options (Checkboxes)

Choose which default features you want enabled for this topic:

- **Add a default subscription** (*Recommended for beginners*):

Leave this **checked**. It will automatically create a pull subscription named <your-topic-id>-sub so you can immediately receive messages without extra setup.

- **Use a schema:**

Leave **unchecked** unless you have already created an Avro/Protobuf schema.

- **Enable ingestion:**

Leave **unchecked** unless you are directly streaming from AWS Kinesis or Azure Event Hubs.

- **Enable message retention:**

Check this only if you want the topic to retain messages for historical playback (e.g., set to 7 days).

- **Export message data to BigQuery / Backup to Cloud Storage:**

Leave **unchecked** for standard topics, or check if you want GCP to auto-create direct streaming pipelines.

Step 5: (Optional) Transforms

- Under **Transforms**, leave it set to default unless you need to run in-flight JavaScript code to clean or modify incoming data.

Step 6: Encryption Settings

- Under **Encryption**, keep **Google-managed encryption key** selected (default).

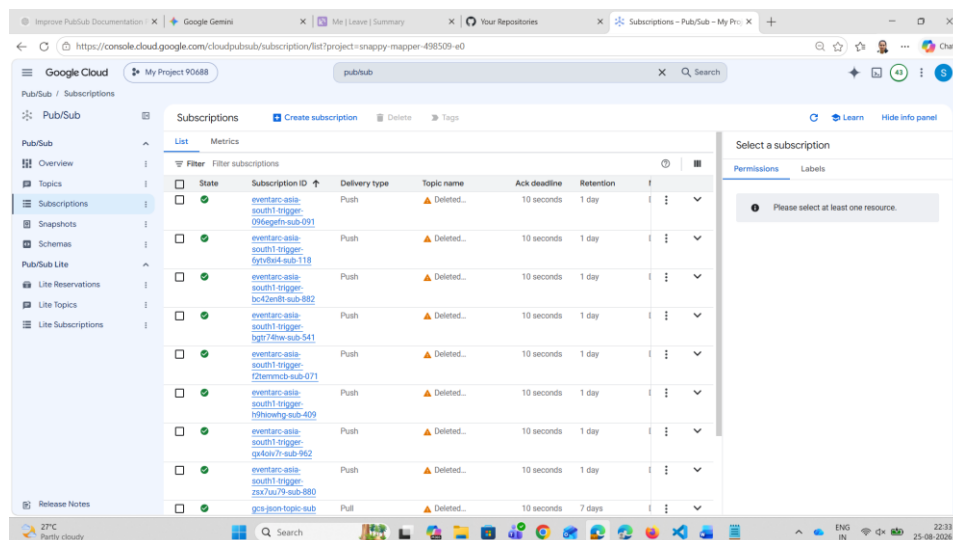
Step 7: Finalize & Create

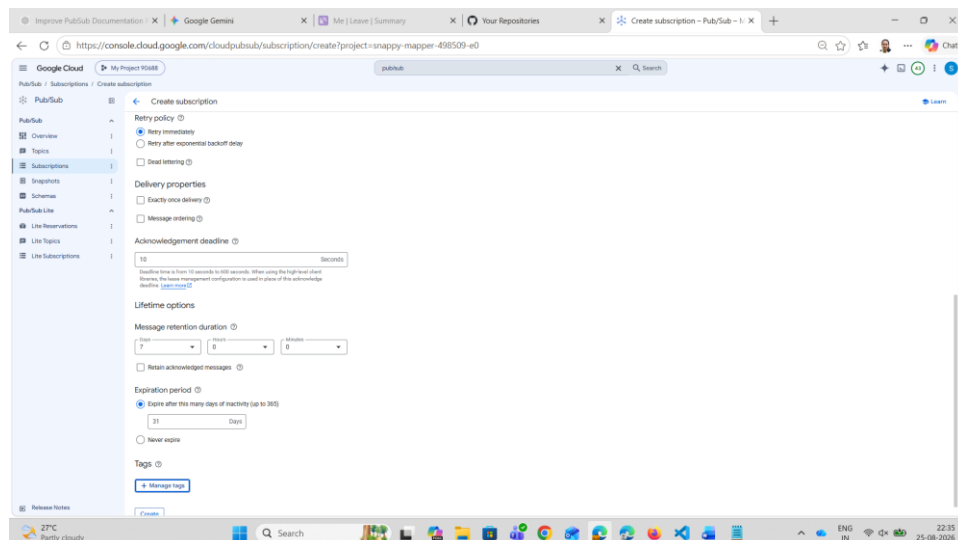
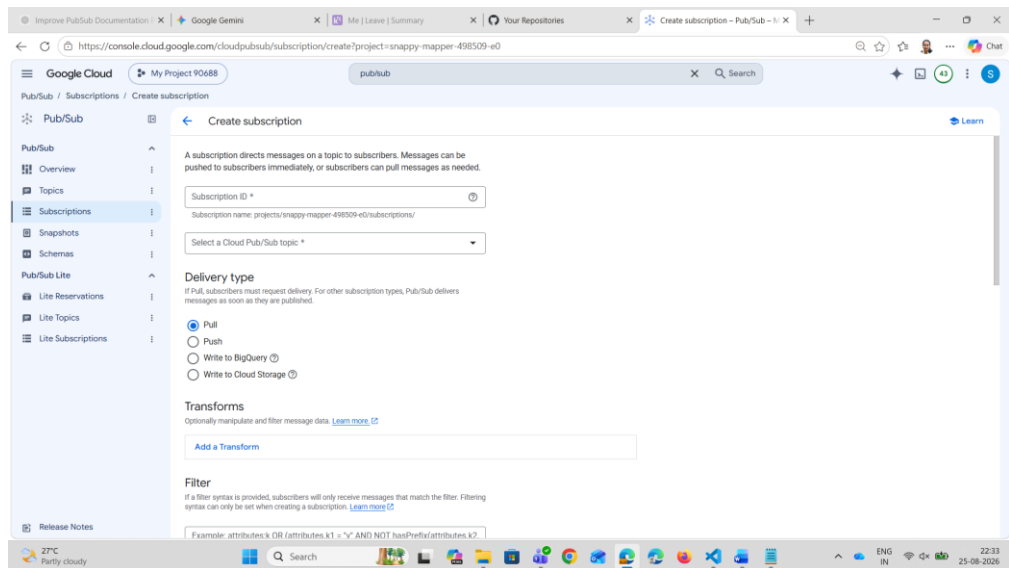
1. Scroll down to the bottom of the page.
2. Click the blue Create button.

Step 8: Verify Your Topic

1. The console will automatically redirect you to the **Topic details** page.
2. Click the **Subscriptions** tab on that page to verify your default subscription was created, or click the **Messages** tab to publish your first test message.

23.Explanation about the UI-Interface in PUB/SUB Subscription:





Identity & Routing

- **Subscription ID:** Unique name of the subscription.
- **Select a Cloud Pub/Sub topic:** Select the topic from which the subscription receives messages.

Delivery Type

- **Pull:** The subscriber requests messages from Pub/Sub.
- **Push:** Pub/Sub sends messages to an HTTPS endpoint.
- **Write to BigQuery:** Sends messages directly to a BigQuery table.

- **Write to Cloud Storage:** Saves messages directly to a Cloud Storage bucket.

Message Filtering

- **Filter:** Allows you to receive only messages that match specific conditions.
- **Example:** Receive only messages where type = "order".
- Messages that do not match the filter are not delivered to the subscriber.

Retry Policy

- **Retry immediately:** Retries the message immediately when processing fails.
- **Exponential backoff:** Waits for increasing amounts of time before retrying.
- **Dead lettering:** Sends messages that repeatedly fail to a Dead-Letter Topic.

Delivery Options

- **Exactly once delivery:** Helps prevent duplicate message delivery.
- **Message ordering:** Keeps messages with the same ordering key in the correct order.

Acknowledgement & Retention

- **Acknowledgement deadline:** The time given to the subscriber to process and acknowledge a message.
- **Message retention duration:** Determines how long messages are kept in the subscription.
- **Retain acknowledged messages:** Keeps already processed messages so they can be replayed later.
- **Expiration period:** Automatically deletes an unused subscription after the configured period.

Subscription List

- **State:** Shows whether the subscription is active and healthy.
- **Subscription ID:** Shows the unique subscription name.
- **Delivery type:** Shows whether the subscription uses Pull, Push, BigQuery, or Cloud Storage.
- **Topic name:** Shows the topic connected to the subscription.

- **Deleted topic:** Means the connected topic has been deleted.
- **Ack deadline:** Shows the time allowed for message acknowledgement.
- **Retention:** Shows how long messages are retained.
- **Message ordering:** Shows whether message ordering is enabled.
- **Exactly once delivery:** Shows whether exactly-once delivery is enabled.
- **Expiration:** Shows when the subscription will expire if it remains unused.

24. Step-by-Step UI Guide to Create a Pub/Sub Subscriptions

Step 1: Open the Subscriptions Page

1. In the Google Cloud Console, type pub/sub in the top search bar and select **Pub/Sub**.
2. From the left sidebar navigation menu, click **Subscriptions**.
3. In the top action bar, click the + Create subscription button.

Step 2: Set Identity & Link Your Topic

1. **Subscription ID:** Enter a unique name for your subscription (e.g., order-worker-sub).
2. **Select a Cloud Pub/Sub topic:** Click the dropdown menu and pick the parent topic you want to consume messages from.

Step 3: Choose the Delivery Type

Select one of the four delivery mechanisms based on how your subscriber works:

- **Pull (Default):** Select this if your application code will connect to Pub/Sub and request messages as it has capacity.
- **Push:** Select this if you want Pub/Sub to send an HTTP POST request to an external webhook or endpoint URL. Enter your HTTPS URL in the **Endpoint URL** field.
- **Write to BigQuery:** Select this to stream messages directly into a BigQuery table without writing consumer code. Choose your Dataset and Table name.
- **Write to Cloud Storage:** Select this to batch and write messages directly to files inside a Cloud Storage bucket.

Step 4: (Optional) Apply Message Filtering

- Locate the **Filter** box.

- If you only want this subscription to receive a subset of messages, enter a filter expression using message attributes (e.g., `attributes.type = "order" AND attributes.priority = "high"`).
- If left blank, this subscription receives 100% of the topic's messages.

Step 5: Configure Failure Handling & Retry Policy

- **Retry immediately:** Pub/Sub redelivers unacknowledged messages as soon as the timeout expires.
- **Retry after exponential backoff delay:** Pub/Sub progressively increases the wait time between retries (e.g., from 10s up to 600s), preventing crashed downstream databases from being overwhelmed.
- **Dead lettering (Checkbox):**
 - Check **Enable dead lettering** if you want to isolate failing messages.
 - Select a **Dead-letter topic** and set the **Maximum delivery attempts** (e.g., 5). If a message fails 5 times, it gets moved out of the main queue.

Step 6: Configure Delivery Properties & Deadlines

- **Exactly once delivery:** Check this if you want Pub/Sub to prevent duplicate deliveries while an ACK is in flight.
- **Message ordering:** Check this if your publisher uses `ordering_key` attributes and messages must arrive in sequential order.
- **Acknowledgement deadline:** Enter the time limit your subscriber has to process a message (default: 10 seconds; can be set up to 600 seconds).

Step 7: Configure Lifetime & Expiration Options

- **Message retention duration:** Set how long unacknowledged messages remain queued (default: 7 days).
- **Retain acknowledged messages (Checkbox):** Check this if you plan to create snapshots or seek back to replay historical data.
- **Expiration period:** Keep **Expire after 31 days of inactivity** (deletes the subscription if unused) or choose **Never expire**.

Step 8: Create and Test

1. Scroll to the bottom and click the blue Create button.
2. Once created, open your new subscription from the list.
3. Go to the **Messages** tab and click the PULL button to inspect incoming messages in real-time.

25.Conclusion

Google Cloud Pub/Sub is a scalable messaging service used for reliable and asynchronous communication between applications and services.

In this documentation, we covered topics, subscriptions, messages, acknowledgements, retries, dead-letter topics, ordering, filtering, schemas, security, monitoring, and GCP integrations.

Pub/Sub is an important service for building real-time data pipelines, event-driven applications, and distributed systems.

Production-Ready Completion Guide

This addendum follows the same progression used in the Managed Airflow guide: introduction → architecture → core concepts → UI/setup → configuration → security → monitoring → troubleshooting → performance/cost → reliability → production design → end-to-end scenario → common mistakes → operational checklist.

A. Standard learning path

Stage	Expected outcome
1. Introduction	Explain the service, problem and role.
2. Architecture	Explain components and flow.
3. Core concepts	Explain hierarchy, execution and key terminology.
4. UI and setup	Create resources and recognize important settings.
5. Configuration	Configure runtime, identity, networking and scaling.
6. Security	Apply IAM, encryption, secrets and least privilege.
7. Monitoring	Use logs, metrics and alerts.
8. Troubleshooting	Move from symptom to evidence to root cause.
9. Performance and cost	Relate design to throughput, latency and cost.
10. Reliability	Handle retries, duplicates and recovery.
11. Production design	Apply environment separation, CI/CD and ownership.
12. End-to-end scenario	Explain the service in a realistic pipeline.

26. Message lifecycle and replay design

A production Pub/Sub design should define how messages are retained, acknowledged, retried, replayed and eventually discarded or dead-lettered.

- Choose topic/subscription retention intentionally.
- Document replay procedures and their impact on downstream systems.
- Do not assume acknowledgement means business processing is permanently complete.

27. Delivery semantics, ordering and exactly-once

Pub/Sub applications must be designed around the delivery semantics required by the workload.

- Use idempotent consumers for duplicate-safe processing.
- Enable ordering only where business logic requires it.
- When using exactly-once delivery features, validate the supported subscriber pattern and operational constraints.

28. Schemas and contract evolution

Schemas provide a contract between publishers and subscribers.

- Use Avro or Protobuf where schema enforcement is valuable.
- Define ownership and compatibility rules.
- Test schema revisions before changing producers.

29. Dead-letter handling and recovery

Dead-letter topics isolate messages that repeatedly fail delivery or acknowledgement.

- Define maximum delivery attempts and required IAM.

- Monitor dead-letter volume.
- Create a replay/remediation process so DLQ messages do not become permanent data loss.

30. Monitoring, quotas and cost

Production Pub/Sub operations should monitor message flow and resource limits.

- Monitor publish/subscribe throughput, backlog and delivery errors.
- Watch quotas and regional/global configuration relevant to the workload.
- Estimate message, storage and network-related costs using current pricing.

31. Common production mistakes

- Assuming exactly-once business effects without idempotency.
- Ignoring subscription retention.
- Using ordering keys unnecessarily and reducing parallelism.
- Creating a DLQ without a replay process.
- Changing schemas without compatibility testing.
- Giving broad project-level IAM.

32. Operational checklist

- Topic and subscription names documented.
- Delivery type and retry policy documented.
- Retention/replay requirements defined.
- Ordering requirement explicitly justified.
- Schema and evolution rules documented.
- DLQ and replay process tested.
- IAM follows least privilege.
- Backlog/error monitoring and alerts configured.
- Quota and cost expectations reviewed.

Official references

<https://docs.cloud.google.com/pubsub/docs/overview>

<https://docs.cloud.google.com/pubsub/docs/subscription-properties>

<https://docs.cloud.google.com/pubsub/docs/schemas>

<https://docs.cloud.google.com/pubsub/docs/dead-letter-topics>

Final learning outcome

After completing this document, the learner should be able to explain Google Cloud Pub/Sub, create or configure its main resources, operate it from the Google Cloud console, troubleshoot common failures, and describe production controls.